

Module 02

Q1 Linear Search (10 Marks)

Linear search, also called **sequential search**, is a simple searching algorithm used to find an element in a list by checking each element one by one until the desired element is found or the list ends.

1. **Simple & Easy** – Linear search is the easiest searching technique that does not require any complex logic or data structures.
2. **Sequential Checking** – The algorithm scans each element in order from the beginning to the end until a match is found or the list is exhausted.
3. **Works on Unsorted Data** – Unlike binary search, which requires sorted data, linear search works on both **sorted and unsorted lists**.
4. **Best Case Complexity: $O(1)$** – If the target element is at the first position, it is found in the first comparison, making it the most efficient scenario.
5. **Worst Case Complexity: $O(n)$** – If the target element is at the last position or not present, the algorithm has to check every element, making it the least efficient scenario.
6. **Average Case Complexity: $O((n+1)/2) \approx O(n)$** – The expected time complexity for random inputs is calculated by taking the sum of all cases divided by the number of cases.
7. **Space Complexity: $O(1)$** – Linear search does not require extra memory other than a few variables, making it a space-efficient algorithm.
8. **Iterative Approach** – It is implemented using a simple loop that iterates over the array, making it easy to understand and code.
9. **Efficient for Small Datasets** – Linear search is practical for small lists but becomes inefficient for large datasets compared to other searching algorithms like binary search.
10. **Implementation Example (Java-like Pseudocode):**

```
for(int i=0; i<n; i++) {
    if(A[i] == k) {
        System.out.println("Element found at index: " + i);
        break;
    }
}
if(i == n) {
    System.out.println("Element not found");
}
```

Q2 What is Search Sort?

Searching algorithms are methods or procedures used to find a specific item or element within a collection of data. They play a crucial role in computer science for tasks such as retrieving records from a database, locating an element in a sorted list, or finding a file on a computer.

Importance of Searching (3 Marks)

1. **Efficiency** – Optimizes program performance by reducing search time.
2. **Data Retrieval** – Helps quickly find and access specific data from large datasets.
3. **Database Systems** – Enables fast querying and retrieval of records in databases.
4. **Problem Solving** – Used in various computational problems like AI and pathfinding.

Q3 What is Insert Sort? (10 Marks)

Insertion Sort is a simple sorting algorithm that **builds the final sorted array one item at a time** by comparing and inserting elements into their correct position. It is an in-place, stable sorting algorithm that works well on **small** or **nearly sorted datasets**.

Key Points with Explanation:

1. **Step-by-Step Sorting** – The algorithm starts with the second element (index 1) and inserts it into its correct position among the previously sorted elements. This process repeats for all elements.
2. **Best Case Complexity: $O(n)$** – If the array is already sorted, each element only requires **one comparison**, making the algorithm run in **linear time**.
3. **Worst Case Complexity: $O(n^2)$** – When the array is sorted in **reverse order**, every element has to be compared and shifted to the beginning, leading to **quadratic time complexity**.
4. **Average Case Complexity: $O(n^2)$** – If the array elements are in random order, the number of comparisons and swaps are **proportional to n^2** , making it inefficient for large datasets.
5. **Time Complexity Analysis:**
 - **Worst Case:** $C_{\text{worst}}(n) = n(n-1)/2 \in O(n^2)$ $C_{\text{worst}}(n) = \frac{n(n-1)}{2} \in \Theta(n^2)$ $C_{\text{worst}}(n) = 2n(n-1) \in O(n^2)$
 - **Average Case:** $C_{\text{avg}}(n) \approx n^2/4 \in O(n^2)$ $C_{\text{avg}}(n) \approx \frac{n^2}{4} \in \Theta(n^2)$ $C_{\text{avg}}(n) \approx 4n^2 \in O(n^2)$
 - **Best Case:** $C_{\text{best}}(n) = n-1 \in O(n)$ $C_{\text{best}}(n) = n-1 \in \Theta(n)$ $C_{\text{best}}(n) = n-1 \in O(n)$
6. **Space Complexity: $O(1)$** – Insertion sort is **in-place**, meaning it does not require extra memory beyond a few auxiliary variables.
7. **Stability:** – It is a **stable sorting algorithm**, meaning equal elements maintain their relative order after sorting.
8. **Binary Insertion Sort:** – A variation of insertion sort where binary search is used to find the correct position for insertion, reducing the number of comparisons to **$O(\log n)$** but keeping **$O(n^2)$ swaps**.
9. **Efficient for Small & Nearly Sorted Arrays:** – Insertion sort is one of the **fastest algorithms** for nearly sorted data, as it minimizes unnecessary swaps and comparisons.
10. **Implementation (Java-like Pseudocode):**

```
for (int i = 1; i < n; i++) {  
    int key = A[i];  
    int j = i - 1;  
    while (j >= 0 && A[j] > key) {  
        A[j + 1] = A[j];  
        j--;  
    }  
    A[j + 1] = key;  
}
```

Q4 what is merge sort? (10 Marks)

Merge Sort is a **divide and conquer** sorting algorithm that **recursively splits** an array into smaller subarrays until each subarray contains a single element. Then, it **merges** them back in sorted order.

Key Points with Explanation:

1. **Step-by-Step Breakdown:**
 - **Step 1:** Recursively **divide** the array into two halves until each subarray has only one element.
 - **Step 2:** Begin **merging** subarrays back together in sorted order.
 - **Step 3:** Continue merging until the entire array is sorted.
2. **Best Case Complexity: $O(n \log n)$** – When the array is already sorted, merge sort still splits it into halves and merges them, leading to **logarithmic depth** and **linear merging**.
3. **Worst Case Complexity: $O(n \log n)$** – Even if the array is sorted in **reverse order**, the number of splits and merges remains the same.
4. **Average Case Complexity: $O(n \log n)$** – No matter the initial order of elements, merge sort consistently **splits and merges in $O(n \log n)$ time**.
5. **Space Complexity: $O(n)$** – Unlike quicksort or insertion sort, **merge sort requires extra space** proportional to the array size, as it creates temporary arrays.
6. **Stable Sorting Algorithm:** – Merge Sort **maintains the relative order of equal elements**, making it **stable** and useful for sorting records with multiple keys.
7. **Divide and Conquer Methodology:** – The algorithm follows the recurrence relation:
 $T(n) = 2T(n/2) + O(n)$
Applying the **Master Theorem**, we get **$T(n) = O(n \log n)$** .

```
def merge_sort(A, lb, ub):
    if lb < ub:
        mid = (lb + ub) // 2
        merge_sort(A, lb, mid)
        merge_sort(A, mid + 1, ub)
        merge(A, lb, mid, ub)

def merge(A, lb, mid, ub):
    L, R = A[lb:mid+1], A[mid+1:ub+1]
    i, j, k = 0, 0, lb

    while i < len(L) and j < len(R):
        if L[i] <= R[j]: A[k], i = L[i], i + 1
        else: A[k], j = R[j], j + 1
        k += 1

    A[k:ub+1] = L[i:] + R[j:]

# Example usage:
arr = [123, 23, 1, 43, 54, 36, 75, 34]
merge_sort(arr, 0, len(arr) - 1)
print(arr)
```

8. **Sorting Example (123, 23, 1, 43, 54, 36, 75, 34):**
 - **Step 1:** Split into [123, 23, 1, 43] and [54, 36, 75, 34].
 - **Step 2:** Further split into [123, 23], [1, 43], [54, 36], and [75, 34].

- **Step 3:** Sort and merge: [23, 123], [1, 43], [36, 54], [34, 75].
 - **Step 4:** Merge sorted halves: [1, 23, 43, 123] and [34, 36, 54, 75].
 - **Step 5:** Final merge to get [1, 23, 34, 36, 43, 54, 75, 123].
9. **Advantages & Disadvantages:**
- **Advantages:** Works well for large datasets, consistent **$O(n \log n)$** time, stable sorting.
 - **Disadvantages:** Uses extra memory, slower than **quicksort** for small datasets.

Q5 What is QuickSort? (10 Marks)

QuickSort is a **divide-and-conquer** sorting algorithm that selects a **pivot**, partitions the array into two halves (smaller and larger elements), and recursively sorts them. It is efficient with an average-case time complexity of **$O(n \log n)$** and is widely used for large datasets.

1. **Divide and Conquer**
QuickSort divides the array into two smaller subarrays around a pivot and sorts them recursively. This makes it highly efficient for large datasets.
2. **Pivot Selection**
A pivot element is chosen, usually the first, last, or a random element. The choice of pivot affects the performance of the algorithm.
3. **Partitioning**
The array is rearranged so that elements smaller than the pivot are placed on the left, and elements greater than the pivot are placed on the right.
4. **Recursive Sorting**
QuickSort is applied recursively to the left and right subarrays until the entire array is sorted. The base case occurs when a subarray has only one element.
5. **Best & Average Case Complexity – $O(n \log n)$**
If the pivot is chosen well, QuickSort consistently divides the array into balanced halves, leading to an optimal time complexity of **$O(n \log n)$** .
6. **Worst Case Complexity – $O(n^2)$**
The worst case occurs when the pivot is always the smallest or largest element, causing **unbalanced** partitions and degrading performance to **$O(n^2)$** .
7. **In-Place Sorting**
QuickSort sorts the array in place without requiring extra memory, making it space-efficient with **$O(\log n)$** space complexity (for recursion stack).
8. **Unstable Sort**
It does not preserve the relative order of equal elements, making it an **unstable** sorting algorithm.
9. **Applications**
Used in databases, search algorithms, and programming libraries (like Python's `sorted()` function, which uses a variant of QuickSort called Timsort).
10. **Comparison with Merge Sort**
QuickSort is generally faster for **in-place** sorting, while Merge Sort is preferred for linked lists due to its **stable sorting** property and better performance on large datasets.

```
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high

    while True:
        while i <= j and arr[i] <= pivot:
            i += 1
        while i <= j and arr[j] > pivot:
            j -= 1
        if i <= j:
```

```

        arr[i], arr[j] = arr[j], arr[i] # Swap elements
    else:
        break

    arr[low], arr[j] = arr[j], arr[low] # Place pivot in correct position
    return j

def quicksort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quicksort(arr, low, pi - 1)
        quicksort(arr, pi + 1, high)

# Example usage
arr = [44, 33, 11, 55, 77, 90, 40, 60, 99, 22, 88]
quicksort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)

```

Q6. Binary Search? (5 Marks)

Binary Search is an efficient **searching algorithm** used to find an element in a **sorted array**. It works by repeatedly dividing the search range in half until the element is found or the range becomes empty.

1. **Divide and Conquer**
 - The algorithm **splits** the array into two halves at each step, reducing the search space **exponentially**.
2. **Requires Sorted Array**
 - Binary Search **only works on sorted arrays**; if the array is unsorted, it must be sorted first.
3. **Comparison with Middle Element**
 - The search key is compared with the **middle element**.
 - If it matches, the search is successful.
 - If it is smaller, search continues in the **left half**; if larger, in the **right half**.
4. **Efficiency – $O(\log n)$**
 - The algorithm reduces the search space by half at each step, making its time complexity **$O(\log n)$** in the best and average cases.
5. **Iterative vs. Recursive Approach**
 - Binary Search can be implemented **iteratively** (using loops) or **recursively** (using function calls).
6. **Best, Worst, and Average Case Complexity**
 - **Best Case: $O(1)$** (Element found at the first comparison).
 - **Average & Worst Case: $O(\log n)$** (Each step halves the array).

Q7 What is Graph traversal (2 Marks)

Graph traversal is the process of visiting all the vertices (nodes) in a graph in a systematic way. It involves exploring or traversing each vertex in a graph by following the edges that connect the vertices. There are two common methods for graph traversal

- ✓ Breadth-First Search (BFS)
- ✓ Depth-First Search (DFS).

Q8 What is Breadth-First Search (BFS) (5 Marks)

Breadth-First Search (BFS) is a **graph traversal algorithm** that explores all nodes at the current depth level before moving to the next level. It uses a **queue** to keep track of nodes and ensures that each node is visited **once**.

1. **Uses a Queue**
 - BFS follows the **FIFO (First In, First Out)** principle using a **queue** to manage the order of traversal.
2. **Starts from a Source Node**
 - The traversal begins from a **starting node**, which is marked as **visited** and added to the queue.
3. **Visits Neighboring Nodes**
 - The algorithm **dequeues** a node, **visits** all its unvisited neighbors, and **enqueues** them.
4. **Processes Nodes Level by Level**
 - BFS explores nodes in order of increasing distance from the starting node, making it useful for **shortest path** problems.
5. **Repeats Until Queue is Empty**
 - Steps 3 and 4 continue until **all reachable nodes** are visited and the queue becomes **empty**.
6. **Applications**
 - BFS is used in **shortest path algorithms, network broadcasting, AI (pathfinding in games), and web crawling**.

Q9 Depth-First Search (DFS) – Definition & Key Points? (5 Marks)

Depth-First Search (DFS) is a **graph traversal algorithm** that explores as far as possible along a branch before backtracking. It uses a **stack** (either explicitly or through recursion) to track visited nodes.

1. **Uses a Stack (or Recursion)**
 - DFS follows the **LIFO (Last In, First Out)** principle using an **explicit stack** or **recursion** to manage traversal.
2. **Starts from a Source Node**
 - Select a **starting node**, mark it as **visited**, and **push** it onto the stack.
3. **Explores Deepest Path First**
 - The algorithm **picks** an unvisited adjacent node, **visits** it, and **pushes** it onto the stack.
4. **Backtracks When Needed**
 - If no adjacent unvisited node exists, DFS **backtracks** by **popping** the last visited node from the stack.
5. **Continues Until Stack is Empty**
 - Steps 3 and 4 repeat until **all reachable nodes** are visited and the **stack becomes empty**.
6. **Applications**
 - DFS is used in **pathfinding, topological sorting, cycle detection, maze solving, and graph connectivity checks**.

Q10 Depth-First Search (DFS) in Graphs (Maybe If they Give For 10 Marks)

1. Definition:

Depth-First Search (DFS) is a **graph traversal algorithm** that explores as far as possible along a branch before backtracking. It can be used to **search, classify edges, and analyze graph properties**.

2. Complexity Analysis:

- **Adjacency Matrix Representation:** $\Theta(|V|^2)\Theta(|V|^2) \rightarrow$ Every vertex checks all others.
 - **Adjacency List Representation:** $\Theta(|V|+|E|)\Theta(|V| + |E|)\Theta(|V|+|E|) \rightarrow$ Each vertex and edge is visited once.
-

3. How DFS Works:

1. **Start at a chosen vertex** and mark it as **visited**.
 2. **Push** it onto the stack (or use recursion).
 3. **Explore its adjacent vertices** recursively, visiting unvisited nodes.
 4. If a **dead-end** is reached, **backtrack** by popping from the stack.
 5. **Continue until all nodes are visited** and the stack is empty.
-

4. Applications of DFS:

- **Graph Connectivity:** Finding connected components.
 - **Cycle Detection:** Identifying cycles via back edges.
 - **Topological Sorting:** Used in Directed Acyclic Graphs (DAGs).
 - **Pathfinding Algorithms:** Maze solving, AI, and state-space searching.
-

5. DFS Algorithm (Pseudocode):

```
DFS(graph, vertex, visited):  
    Mark vertex as visited  
    Process(vertex)  
    for each neighbor in graph[vertex]:  
        if neighbor is not visited:  
            DFS(graph, neighbor, visited)
```

6. DFS Implementation in Python:

```
def dfs(graph, vertex, visited):  
    if vertex not in visited:  
        print(vertex, end=" ") # Process the node  
        visited.add(vertex) # Mark as visited  
        for neighbor in graph[vertex]:  
            dfs(graph, neighbor, visited)  
  
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B', 'F'],  
    'F': ['C', 'E']  
}  
  
visited_set = set()  
print("DFS Traversal:")  
dfs(graph, 'A', visited_set)
```

7. Additional Point

DFS detects cycles by identifying **back edges**—edges that point to an **already visited ancestor** in the recursion stack, indicating a cycle.